DevOps - Notes

Nils Rechberger

2026-04-09

Lecture 01

DevOps Intro

DevOps is a modern approach that integrates software development (Dev) and IT operations (Ops) to improve the efficiency, collaboration, and delivery of software products. It bridges the gap between the development and operations teams by promoting a culture of collaboration and continuous improvement. DevOps aims to automate and streamline the software delivery process, ensuring faster development cycles, better quality, and more reliable releases.

DevOps Lifecycle

Below are the key phases of the DevOps lifecycle:

1. **Plan**: In the planning phase, teams collaborate to define project goals, features, and timelines, often using agile methodologies such as Scrum. Tools like Jira and Trello facilitate task management, aligning team efforts and ensuring transparency across the development process.
2. **Code**: Developers write code using version control systems such as Git to manage changes, ensuring collaboration and proper documentation.
3. **Build**: The code is compiled, and dependencies are managed. Automation tools like Jenkins or Maven ensure that builds happen consistently and efficiently (CI - Continuous integration).
4. **Test**: Automated and manual testing is performed to ensure code quality, functionality, and security. Continuous testing practices help catch regressions and bugs before deployment
5. **Release**: Once the code passes all tests, it is packaged and prepared for deployment. Continuous delivery or continuous deployment (CD) tools like GitHub Actions or CircleCI help automate this step.

6. **Deploy**: The software is deployed to production environments, often using containerization technologies like Docker and orchestration tools like Kubernetes.
7. **Operate**: Monitoring tools like Prometheus or Nagios track the performance and availability of the application, ensuring smooth operations.
8. **Monitor**: Continuous monitoring allows teams to gather feedback, address issues, and optimize performance, feeding back into the next iteration of planning and coding.

> **Note**
>
> These stages form an endless loop of planning, development, and feedback that enables continuous improvement and faster time to market.

Git

A version control system is a system that records the changes made to the code and saves it in a repository. Additionally, it is a collaborative tool that enables multiple individuals to work together on the same code. With the change history we can see who changed what, when and why. There are two types of version control systems:

- Centralized: The code history is stored at a central server every user has access to. Examples are Subversion, Team Foundation Server.
- Distributed: Every user has a local copy of the code history. Examples are Git, Mercurial.

Git Configuration

Before using git, there are a few configurations that have to be made. The settings are saved in .gitconfig files. Configurations can be set on three levels

- System - Settings apply to all user on that system (`.gitconfig` in a system folder like `root/etc`)
- Global - Settings apply to all repositories of the current user (`.gitconfig` file in the user's home directory)
- Local - Settings only apply to the current repository (`.gitconfig` in the project root directory)

```
# See all configurations
git config --list
git config --list --local
git config --list --global
git config --list --system # if any
```

```
# See a specific configuration
git config user.name

# More info
git config --help # extended
git config -h # short
```

Create a Git Repository

Create Remote Repository & Clone

1. Go to your global repository (in our case GitHub) and create a repository
2. Clone

```
git clone https://github.com/*your-github-name*/*your-repo*.git # using https
git clone git@github.com:*your-github-name*/*your-repo*.git # using ssh
```

Create Locally & Add Remote Repository

```
# go to the project directory and run
git init

# add a remote (Note: Remote has to be set up beforehand)
git remote add origin https://github.com/OWNER/REPOSITORY.git # using https
git remote add origin git@github.com:OWNER/REPOSITORY.git # using ssh
```

Commit Workflow

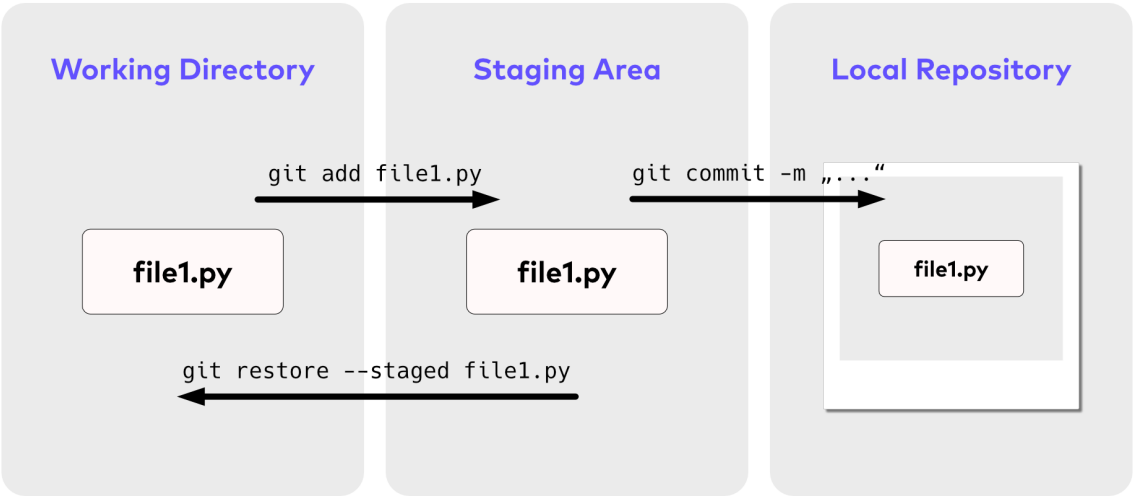

Figure 1: Git Stages

Add the Changes to the Staging Area

```
# Add files
git add . # add all changes

# Remove files
git restore --staged *filename* # "remove" file from the staging area
git rm --cached *filename* # use this when you don't have any commits yet
```

Commit

Every commit (snapshot) gets a unique id and contains a complete snapshot of the project. Additionally, it also contains information about who created the commit and when it was created.

```
git commit -m "Your commit comment"
```

Push to Remote Repository

```
git pull # pull changes from remote first, then push
git push
```

Best Pracitices

- **Use meaningful messages**: This is like a process documentation. Try to omit "Fixed bugs" or "Made changes"
- **Commit on a regular basis**: Commit when you've completed a single logical change, even if it's small.
- **Don't make your commits too big**: Try to break down a big change into small commit packets.
- **A commit should represent only one particular change**: Don't group together different "logical" changes.

Helpful commands

```
git status
git diff . # See difference between working version and staged/committed version
git diff *filename* # Same, but for a specific file
git diff --staged . # See difference between staged and committed version

git log
git log --oneline
```

Ignore Files and Directories

.gitignore

Git offers the option to specify which files and folders should be ignored. This "blacklist," is saved in a file named .gitignore, located in the root directory of your project.

```
venv/ # ignore all files and directories inside the venv folder
somedir/file.txt # ignore a specific file
*.txt # ingore all .txt files
!file1.txt # "!" is an exception --> all .txt files are ignored, except file1.txt
```

Git History

Each commit has its own ID (commit-id) and each commit is pointing to the previous commit. All commits created so far belong to the master branch (also called main branch), which is the main line of work. Git uses a pointer for every branch, so MAIN points to the last commit of the current work line. As new commits are created, the MAIN pointer moves forward. When we start using other branches, git will use the HEAD pointer, to keep track of which branch we're currently working on. Same as the MAIN pointer, the HEAD pointer points to the latest commit. HEAD is a reference/alias to the last commit.

The Log Command

`git log` shows all commits and changes of the current repository.

```
git log --oneline  # only display comments (can be combined with others)
git log --stat # see all files that have been changed with the commits
git log --patch # see how the code changed (like git diff)
```

View Previous Commits

Use git show to see what changes were made with a specific commit.

```
# See changes made with a commit
git show d15728b # show commit with id 'd15728b...' (the whole number is not needed, as long
git show HEAD~1 # show 2nd last commit
```

Compare Two Commits

```
it diff HEAD~2 HEAD # see diff between two snapshots
git diff HEAD~2 HEAD file1.txt # see diff only for file1.txt
```

Checking Out a Commit

Sometimes we want to see the code version as it was at a certain commit. Therefore, we can check out a commit, which will restore the working directory to the snapshot version of this commit. So the working directory will exactly look like in an earlier point in time.

```
git checkout d15728b # commit id
git checkout HEAD~3
git checkout main # return to the latest commit
```

What happens in the background is that the HEAD pointer is now pointing to a previous commit. This brings us in „detached head“ state: The HEAD pointer has been moved to an earlier commit. In this state, no commits should be made, it is only to explore the code.

Programming Styles

Notebook-Style Programming

Data scientists often start with notebook-style programming, particularly in tools like Jupyter Notebooks, for several reasons.

- **Interactivity and Exploration**: Notebooks allow for an interactive and exploratory coding experience.
- **Documentation and Explanation**: Notebooks integrate code with documentation in a coherent narrative
- **Visualization Capabilities**: Data scientists often need to visualize data to understand patterns, trends, and outliers. Notebooks provide seamless integration of code and visualizations.
- **Experimentation and Prototyping**: Notebooks are well-suited for prototyping and experimenting with different approaches.

There are also downsides data scientists should be aware of.

- Version Control Challenge
- Hidden State and Execution Order
- Lack of Code Modularity

> **Note**
>
> You can also combine notebooks and scripts. Start separating recurring functions in scrips and modules.

Procedural Programming

Procedural programming is a programming paradigm built around the idea that programs are sequences of instructions to be executed. They focus heavily on splitting up programs into procedures, named sets of instructions, analogous to functions. A procedure can store local

data that is not accessible from outside the procedure's scope and can also access and modify global data variables.

Object Oriented Programming (OOP)

Object-oriented programming (OOP) is a method of structuring a program by bundling related properties and behaviors into individual objects. For example, an object could represent a person with properties like a name, age, and address and behaviors such as walking, talking, breathing, and running.

Classes

Classes allow you to create user-defined data structures. Classes define functions called methods, which identify the behaviors and actions that an object created from the class can perform with its data.

> **Note**
>
> A class is a blueprint for how to define something. It doesn't actually contain any data. While the class is the blueprint, an instance is an object that's built from a class and contains real data.

4 Main Pillars of OOP

- **Encapsulation**: Hiding data and behavior within a class, making it inaccessible to other classes or code.
- **Abstraction**: Simplifying complex systems by breaking them down into smaller, more manageable parts.
- **Inheritance**: Inherit properties and behaviors from a parent class.
- **Polymorphism**: Behave differently based on the context in which it is used.

Functional Programming (FP)

At its core, Functional Programming (FP) is a programming paradigm that treats computation as the evaluation of mathematical functions and avoids changing-state and mutable data.

Core Concepts of Functional Programming

1. **Pure Functions**: Pure functions are functions that have no side effects.
2. **Immutable Data**: Do not modify data in-place.
3. **Lazy Evaluation**: Only evaluate as expression when we need to operate on it.

Lecture 02

Branching

Branches allow us to diverge from the main line of work and to develop something in isolation. We can work on various new features, without messing up the main line of work. Git uses the HEAD pointer to point to the branch.

```
git branch # list all local branches
git branch -a # list all branches (local and in repository)
git switch <name> # switch between branches
```

Merging

Merging is the act of bringing to branches together. There are two types of merges.

Fast-Forward Merge

When the two branches have not diverged from each other and there is a direct linear path from the target branch to the source branch, the merge is pretty simple. Just bring the pointer of the target branch to the latest commit of the source branch.

> Note: By default, git will first try to do a Fast-Forward merge. If this is not possible, a 3-Way merge is executed.

3-Way Merge

The two branches have diverged from each other. A merge creates a new commit (merge commit) that combines the changes from the two branches. It is called 3-Way because it is based on three commits: The common ancestor and the latest two commits of the branches.

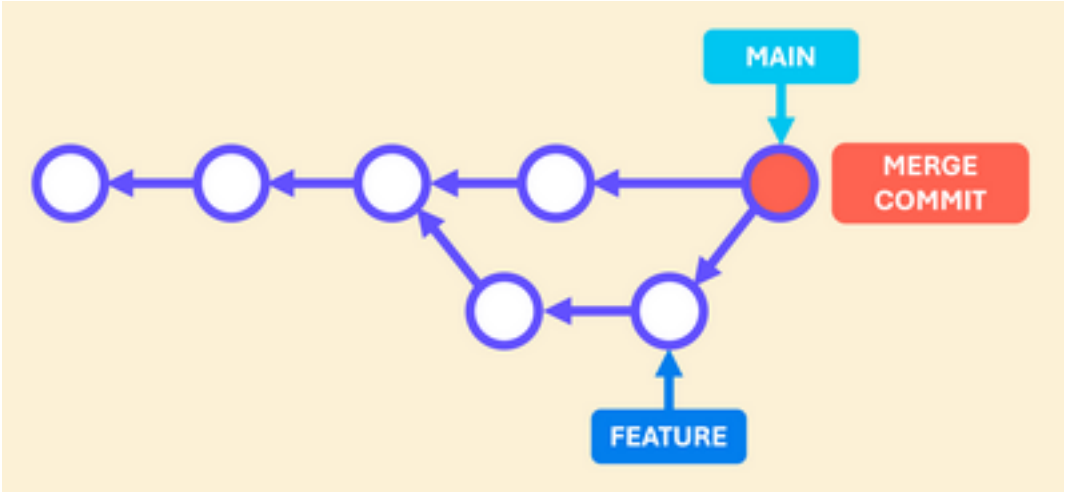

Figure 2: 3-Way Merge

```
git switch main # go to main first
git merge branch-name
```

Merge Conflicts

When git encounters a problem during merging, we get a merge conflict. This happens, for example, when the same file has been changed in both branches. Git then stops the merge process and asks for support to solve the conflict. To solve the merge conflict, decide which code you want to keep and delete the rest.

> Note. Use `git merge --abort` in case you want to abort the merge.

The Problem of Merging

The problem of merging with merge commits is the „non linear" history. As soon as we have multiple branches and merge commits, the history gets hard to read. By rebasing the „base" commit of the branch to the current commit, we can use a Fast-Forward merge and get a linear commit history.

```
git switch branch-name
git rebase main
```

Now the feature-branch originates from the latest commit in the main branch.

> **Caution**
>
> Be causious with rebasing, because it rewrites history! Under the hood, git has to create new commits (since commits cannot be modified) and overwrites the previous ones.

Best is to use rebasing only for local branches. As soon as the branch has been pushed to remote, don't use rebasing anymore. Except you know exactly what you're doing.

Fetching, Pulling & Pushing

The Remote Repository

Besides recording code changes, git allows us to collaborate. To share local changes with others, there needs to be a remote repository every contributor has access to. The remote repository can be a private server or a cloud service. Most common is GitHub.

```
git remote # show all remote repositories (usually just one, "origin")
git remote -v # more details
```

In order to keep track of the branches in the remote repository, git uses remote tracking branches. Despite it's called branch, it's not like a regular branch (we cannot check it out, for example). They're just there to let us know where the remote pointers are pointing to.

Fetching

The remote repository is not connected to the local repository, in the sense, that any changes to the remote are NOT automatically tracked in the local repository. Changes have to be downloaded with the fetch command. The remote changes are now locally saved (with the origin/main pointer pointing to it). However, the local main pointer is not updated by the fetch command. Therefore, we have to do a merge in the same way as we would merge a branch (git merge origin/main).

> **Note**
>
> A shortcut to fetch and merge changes from the remote repository is git pull (pull = fetch + merge).

Pushing

Upload the local changes to the remote repository.

```
git push origin main
git push # 'origin' is optional, if there is just one remote. 'main' is optional when we are
```

Sharing Branches

Every local branch may be linked to a remote branch. This „upstream“ branch tells git, where to push the changes to.

```
# push a new branch (create an upstream branch)
git push -u origin branch-name

# delete a remote branch
git push -d origin branch-name
```

Pull Requests

When we collaborate with others on a project, it is good practice to use pull request for merging feature/bugfix branches. Instead of just merging the changes into the main branch, we would like other to review and comment our code before merging. In the case where the reviewer has requested changes, the PR-owner can update the pull request by adding commits to the branch.

Forking

When you want to contribute to a open-source project, you usually don't have push access to the repository. So, you cannot create branches for a pull request. In this case, you can fork the repository. By forking a repository, a complete copy of the repo is created in your account. It allows you to make changes to the project independently and experiment without affecting the original repository. Once you've made changes in your forked repo, you can submit a pull request to suggest those changes be merged into the original project. This is a common way to collaborate on open-source projects.

Syncing a Fork

Despite having a complete copy of the original repository, your forked repo is still connected to the original one and you have several options to synchronize, when the original repo changed.

> Note: The simplest way is to use the "Sync fork" button in the GitHub web UI.

Tags

Annotated & Lightweight Tags

A best practice is to consider Annotated tags as public, and Lightweight tags as private. Annotated tags store extra meta data such as: the tagger name, email, and date. This is important data for a public release. Lightweight tags are essentially 'bookmarks' to a commit, they are just a name and a pointer to a commit, useful for creating quick links to relevant commits.

```
# Create Lightweight Tag
git tag tagname # set tag on current commit
git tag tagname commit-id # set tag on specific commit

# Create an Annotated Tag
git tag -a tagname -m "Message"

git tag # show all tags
git tag -n # show tags with their messages

# Delete Tag
git tag -d  # delete tag
```

Undo Changes

Restore

Changes made to the files in the working directory or the staging are can be reverted. This command takes the copy from the „next" environment (working dir - staging area - commit) and replaces the current version.

> Note: When nothing has been staged (git add), then staging area = last commit.

```
git restore file1.txt
git restore .

# restore files in the staging area by using the latest commit version
git restore --staged file1.txt
```

Clean

However, git restore does not remove untracked files. To overwrite everything use git clean.

```
git clean -f # force is required
git clean -fd # remove whole directories
```

Restoring

Assume we've deleted a file and removed it from git many commits ago. So, git restore alone won't bring it back.

```
# Step 1: Find the last commit of that file (or any other commit containing the file)
git log -- file1.txt

# Step 2: Restore the file from this commit
git restore --source=d15728b file1.txt
```

Undo Commits/Merges

> **Important**
>
> The Golden Rule: Don't change the public history! If your commit is already pushed to the remote repository, use git revert instead.

Reset

With git reset we can delete commits. This includes also merge commits, so that we can undo a merge. What reset actually does is to set the HEAD and MASTER pointer to a previous commit. The latest commit is still there, but it is a lose end now. Git will delete lose ends automatically during cleanup.

```
# Just set HEAD and MASTER pointer to this commit (working and staging dir stay the same)
git reset --soft commit-id
git reset --soft HEAD~1
```

Recovery

What when we accidentally reset the commit? As long as git hasn't deleted the lose ends, we can recover the state.

```
# Step 1: Find out the commit-id of the "lose end"
git reflog # shows the "reference log" = the log of how the HEAD pointer was moved

# Step 2: Recover
git reset --hard commit-id # commit id of "lose end"
git reset --hard HEAD@{1} # alternatively use this
```

Revert Commits/Merges

When the changes have already been pushed to the remote, it is not a good idea to delete the commit. Instead, create a commit that reverts the commit.

```
git revert HEAD # or commit-id

# In case of merge commits
git revert -m 1 HEAD # 1st parent
```

Scrum

Scrum is an agile project management framework. It was originally developed for software development but has since been applied to other fields. The process is divided into small, manageable units called "sprints", typically lasting 2-4 weeks. The team meets daily to discuss progress and roadblocks. At the end of each sprint, the team reviews the work completed and plans for the next sprint. The goal is to create learning loops to quickly gather and integrate customer feedback. Over the next few chapters we will discuss Scrum in more detail.

Product Owner

The main job of the Product Owner is to bring the product vision to life. This person represents the business or the customer community and is responsible for working with the customers to determine what features will be in the product release. The Product Owner defines the priorities of the product backlog, ensuring that the development team is working on the most valuable features. They communicate the vision of the product to the development team, negotiates the scope and timeline and is responsible for the profitability of the product.

Scrum Master

The Scrum Master is the Scrum expert, who coaches the developers, the product owner and the business on the Scrum process. They are responsible for managing the process of how information is exchanged. Unlike a traditional project manager, the Scrum Master does not provide day-to-day direction to the team and does not assign tasks to individuals. Instead, the Scrum Master guides, coaches, and supports the team in following and improving the Scrum practices. They also help remove any obstacles or impediments that the team might be facing in their progress.

Development Team

The Development Team is responsible for delivering potentially shippable increments of the product at the end of each Sprint. It is usually a self-organized team, which decides who does what, when and how. Members participate in all Scrum events, including the Daily, Sprint Planning, Sprint Review and Sprint Retrospective. The Development Team is cross-functional, meaning it includes all the expertise necessary to deliver the product increment, and it's accountable as a whole for the success of each Sprint.

Product Backlog

The Product Backlog is a list of todo-tasks, called Product Backlog Items (PBI). The backlog is maintained by the Product Owner, who adds new tasks and prioritizes existing ones on a regular basis. Prioritization of tasks may change, as customers requirements and the market situation evolve over time.

Product Backlog Item (PBI)

A PBI is a single element inside the Product Backlog. It is a todo-task. Each PBI should have Acceptance Criteria. They are a formal description of what is required for a task to be

completed/done. Only when all acceptance criteria are fulfilled, a task may be considered as complete/done.

User Login Feature
As a returning user, I want to log in to my account using my email and password, so that I can access my personalized dashboard.

Acceptance criteria:

- User with valid credentials should be logged in and redirected to the dashboard.
- User with incorrect credentials should see an “Invalid email or password” error.
- Invalid email format should trigger a “Please enter a valid email address” error.
- Submitting without a password should trigger a “Password is required” error.
- Clicking “Forgot Password?” should redirect the user to the password reset page.

Figure 3: Example PBI

Sprint Backlog

The Sprint Backlog is a subset of the Product Backlog, which contains all tasks that were selected by the development team for implementation in the current sprint cycle. It is defined in the Sprint Planning, but it may change during the sprint.

Increment

Is the usable end-product from a sprint. This end-product does not have to be a new release, it is rather a stepping stone towards the product goal. Multiple Increments may be created within a Sprint. They can result from a single task or multiple tasks together, but only if the tasks are completed.

Sprint Planning

Every sprint starts with the Sprint Planning. During this meeting, the product owner and the development team decide together which task should be done in the sprint (scope) by moving the tasks from the Product Backlog to the Sprint Backlog. Moreover, a Sprint Goal is defined. The Product Owner guides through the meeting. At the end, each scrum member needs to be clear what the increment of the sprint will be.

> Note: The Product Owner leads the Sprint Planning.

Daily Scrum

As the name suggests, the Daily Scrum meeting is held every day, typically at the same time and place. It should not last longer than 15 minutes. During the Daily, each team member provides updates on what they accomplished the previous day, what they plan to work on that day and any obstacles or issues they are facing. The purpose of this meeting is to promote communication, identify potential issues early and ensure the team is aligned and moving towards the sprint goal.

> Note: The Scrum Master leads the Daily Scrum.

Backlog Grooming / Refinement

Updating the product backlog regularly, helps prioritizing the right work. During this ceremony the Product Owner together with Development Team and Scrum Master review and discuss current and new backlog items. Delete items that aren't relevant anymore, add new ones and divide tasks that are too large into smaller items.

> Note: The Product Owner leads the Backlog Grooming / Refinement.

Sprint Review

The Sprint Review is a collaborative event held at the end of each sprint, bringing together the Scrum Team and stakeholders to inspect the increment and discuss progress. Rather than a formal product demonstration, it is an open conversation — covering what was completed, what wasn't, obstacles encountered, and what that means for the path ahead. The entire Scrum Team shares ownership of the event. The Product Owner clarifies which items are done and facilitates discussion around the product backlog, while the Scrum Master supports the team in keeping the event focused and within its timebox. The Sprint Review is one of the key opportunities for transparency and adaptation in Scrum. Its output is a revised product backlog that reflects new insights, stakeholder feedback, and any shifts in priority.

> Note: The Product Owner leads the Sprint Review

Sprint Retrospective

The Sprint Retrospective is usually held after the Sprint Review and before the next Sprint Planning. It is a chance for the Scrum Team to reflect on the past sprint, discussing what went well, what didn't, and how they can improve in the next sprint. It fosters a culture of continuous improvement, encouraging the team to enhance their work processes and efficiency. The Scrum Master leads the meeting, ensuring a safe and open environment for the team

to share their thoughts and ideas. The outcomes of this meeting often lead to actionable commitments for improvement in the next sprint.

Note: The Scrum Master leads the Sprint Retrospective.

Kanban

Kanban is more a visual project management framework that centers around the Kanban-Board and the concept of "flow". The main goal is to identify potential bottlenecks in the process and fix them so work can flow through it cost-effectively at an optimal speed and throughput. In contrast to Scrum, it doesn't use sprints, instead, tasks flow continuously. Kanban is commonly used in areas where work is less "plannable" (e.g. a service line).